Cochinae — frame-level possession from automatic tracking

Team: Cochinae (Sebastian Dulong Salazar, Santiago Fernández del Castillo, Ulises Quevedo)
Challenge: More than a Hack 2027, Phase 1 — Ball Possession (FC Barcelona vs Athletic Club)
Code: this repository. `./reproduce.sh` regenerates `Cochinae.csv` byte for byte.

---

1. Approach in one paragraph

Possession is decided by two kinds of evidence that fail in different places. The **ball** is precise when it is tracked and absent when it is not. The **shape of the two teams** is always there but is blurry and late, because teams reorganise *after* the ball changes hands. We turn both into per-frame log-likelihoods for `H`, `A` and `D`. We then decode the whole half at once with a Viterbi pass that charges a fixed cost for every change of possession and every stoppage, so the output is a set of whole possessions rather than a flicker of frames. A round of self-training sharpens the team evidence. Finally, a boundary-refinement step moves each stoppage edge onto the moment the players' speed shows a restart.

No stage is trained on hand labels. The team classifiers are self-supervised from the tracking itself. The public 5-minute ground truth is used for one small model (the dead-ball logistic regression, 8 coefficients), and inside that window its predictions are out-of-fold.

2. Pipeline

stage	what it does	trained on
`load.py`	Parses ATD and Smart Tagging into arrays (metres, video-frame index).	—
1. shape	Gradient-boosted classifier, P(away has the ball) from player geometry only: team centroids, depth, width, spread, forward velocity, speeds, and their differences, each with ±0.5 s and ±2 s rolling context. All oriented so "home attacks +x" in both halves.	**Self-supervised**: frames where the ball is visible, one player is within 1.5 m and the other team is ≥1 m further away (35k frames). Out-of-fold by 20,000-frame time blocks.
2. touch	Where the ball is visible and someone is within 3 m: logit += 1.2 × (distance advantage in m, clipped ±3).	—
3. dead	Logistic regression on 8 physical cues: player mean speed (±2 s), max speed (±3 s), share of frames the ball travels >3 m/s, share it is still, ball visibility, ball outside the pitch (±4 s), players within 5 m of the ball, ball distance to the nearest line.	Public window (4-fold out-of-fold inside it).
4. decode	Viterbi over H / A / D per half. Switching team costs 30 nats; entering or leaving D costs 20; D is a chain of 50 sub-states, so a stoppage lasts ≥2 s.	—
5. self-train	A second classifier using shape **plus** ball features (distances per team, oriented ball position and velocity, visibility) learns from the decoded labels, skipping ±1 s around every decoded change. Then we decode again.	Self-supervised (decoded labels), out-of-fold.
6. refine	Each decoded restart moves to the slowest moment of the players in the 6 s before it. Each stoppage moves to the best "fast → slow" changepoint of player speed.	—

Why these choices

- **Shape before ball.** On the public window, team shape alone puts 87.5% of live frames on the right team. Nearest-player-to-ball gets 77.8%. Self-training with ball features raises it to about 91%.

- **The missing ball is mostly dead ball.** The ball is absent from 41% of graded frames, but in long blocks: the median gap is 10.7 s. 93% of those frames are decoded as `D`. Inside live play the ball is usually there. That is why the shape model is the safety net and not the main signal during play.
- **A minimum stoppage length is needed for correctness, not only for looks.** Without it, a 1-frame `D` costs two dead-ball transitions. That is cheaper than one team switch, so the decoder swaps teams through a fake stoppage.
- **Lag correction, measured without labels.** Shape evidence correlates best with ball proximity when it is shifted about 30 frames earlier (cross-correlation over the whole match). The self-trained model's best shift is about 5 frames. We apply both shifts.
- **Restarts sit at the players' slowest moment.** Averaged over the public restarts, mean player speed bottoms out at the restart frame (≈0.9 m/s) and climbs after it. The dead-ball probability fades over 2–3 s, so the decoded restart lands late. The refinement step cut the median restart error from 75 to 34 frames.

3. Possession rules we implemented

- **Live ball always belongs to a team.** There is no "contested" state: a loose ball stays with the team that last had it until the evidence for the other team outweighs the 30-nat switching cost. That cost corresponds to about 1–2 s of clear evidence. So deflections and single touches without control do not change possession, while a recovery that the receiving team keeps does.
- **Short possessions are sacrificed on purpose.** Possessions under about 1 s (a touch that is immediately lost) are mostly not reported. The ranking metric charges a split possession twice, so missing a 1 s spell is the cheaper error.
- **Stoppages last at least 2 s** and are only entered when the physical cues agree. Dead-ball ownership is never assigned, as the rules ask.
- **Restarts** begin when the players start moving after their slowest moment, not when the dead-ball probability falls.

4. Assets used and dropped

asset	used?	notes
Tactical ATD (players + ball)	**yes**, everything is built on it	
Public 5-min ground truth	dead-ball model + sanity checks	never used to train the team models
Smart Tagging	**dropped**	Its team tags put the right team in possession on only 70% of public live frames, worse than team shape. Its set-piece intervals added nothing to the dead-ball model once the physical cues were in.
Broadcast ATD	**tried, not in the final pipeline**	`align.py` aligns it to the tactical timeline: the offset is near-constant per half (≈4,015 and ≈26,921 frames) and 95% of coarse matches agree within 1 m. But it supplies a ball on only 8.5% of the frames where the tactical ball is missing, because both cameras lose the ball in the same moments.
Tactical video + homography	**tried, dropped**	`cochinae/video_ball.py` runs a detector only where the ATD has no ball and play may be live. Checked on 2,000 frames where the ATD does have the ball: COCO YOLOv8s at confidence ≥ 0.6 fires on 22% of frames and only 50% of those are within 2 m. FootAndBall fires on 29% and under 5% are within 2 m. The ball is 3–5 px in this full-pitch shot. Only about 5% of graded frames are live with no ball, so the gain is capped. Full write-up in paper/PAPER.md.

Tried and dropped, besides the above: a "last controller keeps it" rule (best public spellF1 0.59 using our own dead-ball mask, vs 0.60 for the HMM with the same mask), and snapping team switches to the changepoint of ball proximity (it made timing worse: in the automatic tracking the true switch is not where the new team first reaches the ball).

5. Results

Public 5-minute window (17 possessions), scored with the official `run_evaluation.py`:

submission	macroF1	spellF1	possessions
Cochinae	**0.867**	**0.542**	14 (truth: 17)
baseline_smooth125	0.581	0.424	16
baseline_smooth50 (the published floor)	0.621	0.326	26

The official scorer's bootstrap says gaps under about 0.41 spellF1 are noise on this window, so we used it to check correctness rather than to choose settings. Hyperparameters were set from physical reasoning and from full-match plausibility, which needs no labels. On the full match the model reports 265 possessions, mean 12.4 s, and 45% of graded frames dead. Those figures are consistent with the 14.2 s mean possession in the public window and with typical ball-in-play time in LaLiga.

Robustness checks (no ground truth needed)

The public window is too small to choose settings, so we checked stability on the full match instead. We perturbed one parameter at a time and scored each perturbed run against the reference run with the official metric.

perturbation	agreement with reference (spellF1)
switching cost 20–40, self-training switching cost 20–40	0.89–0.99
dead-ball entry cost 10–30, minimum stoppage 1–4 s	0.92–0.94
dead-ball regularisation C 0.005–1.0 (200× range)	0.88–0.92
restart search window 4–8 s, self-training lag 0–10 frames	0.96–0.99
shape lag 0–45 frames, proximity weight 0.6–2.0	0.83–0.90 (the most sensitive)

For the two most sensitive parameters we ran a 3×3 grid and picked the **medoid**: the output that agrees best with the other eight. It is the setting we already use (weight 1.2, lag 30), with a mean agreement of 0.875. The chosen point sits in the middle of a flat region, not on an edge tuned to 17 public possessions.

6. Limitations and what we would do next

- **Short possessions** (< 2 s) are mostly merged into their neighbours. A touch detector trained on corrected data would recover them.
- **Team-switch timing** still errs by about 0.4 s at the median when we find the switch, and about 5 of 12 public switches are not matched within 6 s.
- **The dead-ball model is fit on 4 stoppages.** It uses few, physical features to limit overfitting, but it is the stage most exposed to the next match being different.
- **Next:** fine-tune a ball detector on this camera, using tracked ball positions projected through the homography as automatic labels (off-the-shelf detectors failed, see above). Use the broadcast alignment to recover the ball during replays. Learn the switching cost per field zone: switches are more frequent in midfield duels than in a team's own third.

7. Reproduction

```
 python -m venv .venv && source .venv/bin/activate
pip install -r requirements-cochinae.txt
MTAH_DATA_DIR=/path/to/bundle ./reproduce.sh
```

The script deletes any cached intermediate results, parses the data (`cochinae/load.py`, about 10 s), runs the model (`cochinae/model.py`, about 2.5 min on an M-series laptop), validates the file and prints its SHA-256. Everything is deterministic: the classifiers have fixed seeds and no early-stopping splits, and caches are keyed by a content hash. Two from-scratch runs give the same file. The output is `evaluation/results/Cochinae.csv`.

Code map: `cochinae/load.py` (parsing), `feats.py` (shape and ball features), `model.py` (stages 1–5), `refine.py` (stage 6), `evalu.py` (in-process scorer), `align.py` and `broadcast.py` (the broadcast experiment).